

Appunti di Algoritmi Distribuiti e Reti Complesse

Zbirciog Ionut Georgian

15 marzo 2026

Indice

1	Modellazione Del Sistema	2
1.1	Definizioni Elementari	2
1.2	Assiomi e Restrizioni	5
1.2.1	Assiomi	5
1.2.2	Resitrizioni	6
2	Broadcast	8
2.1	Algoritmo Flooding	9
2.1.1	Complessità Temporale	11
2.1.2	Stati ed Eventi	12
2.1.3	Lowerbound alla Message Complexity	13
3	Spanning Tree Construction	15
3.1	Single Source SHOUT Protocol	16
3.1.1	Terminazione	18
3.1.2	Correttezza	18
3.1.3	Message Complexity	19
3.2	Single Source SHOUT+ Protocol	21

Capitolo 1

Modellazione Del Sistema

Definizione 1.0.1 (Sistema Distribuito). Un *sistema distribuito* è un insieme **entità** (dette anche **nodi**, **agenti**) che eseguono un algoritmo cooperando tra loro, comunicando e coordinandosi per raggiungere un obiettivo comune. Ogni nodo ha una propria memoria locale e può comunicare con altri nodi attraverso un canale di comunicazione. Internamente, ciascun nodo possiede un clock locale, che scandisce il tempo per le comunicazioni e le operazioni. I vari clock dei nodi non sono necessariamente sincronizzati tra loro, e possono procedere a velocità diverse. Inoltre, i nodi possono subire guasti, come crash o malfunzionamenti, che possono influenzare il comportamento del sistema.

Tale ambiente distribuito viene modellato mediante grafo $G = (V, E)$, dove V ($|V| = n$) è l'insieme dei vertici (nodi) e E ($|E| = m$) è l'insieme degli archi (interconnessioni). In particolar modo, data una rete costituita da agenti e interconnessioni tra di essi, il grafo sottostante ha un vertice per ciascun agente del sistema e un arco tra due nodi per ogni interconnessione tra i corrispettivi agenti.

1.1 Definizioni Elementari

Definizione 1.1.1 (Entità). Un'ambiente distribuito è costituito da un insieme $\mathcal{E}$ di entità. Ogni nodo $x \in \mathcal{E}$ della rete, ha delle capacità computazionali, le quali sono definite come un insieme di operazioni possibili, tra le quali

- **Comunicazione**: capacità di inviare e ricevere messaggi ad altri nodi della rete.
- **Local storage & processing**: capacità di memorizzare e elaborare informazioni localmente.
- **Clock**: capacità di tenere traccia del tempo, impostando un clock locale che scandisce il tempo per le comunicazioni e le operazioni. I vari clock dei nodi non sono necessariamente sincronizzati tra loro, e possono procedere a velocità diverse.
- **Modificare registri locali**: capacità di modificare i propri **registri locali**, che rappresentano lo stato interno del nodo.

Definizione 1.1.2 (Stato). Definiamo Σ come l'insieme di tutti gli stati che può assumere un nodo $x \in \mathcal{E}$. L'insieme degli stati è definito a priori e deve essere un insieme finito. Ad

esempio,

$$\Sigma = \{\text{idle, computing, waiting, } \dots\}$$

È importante notare che ad ogni istante di tempo t , un nodo x si trova in uno stato specifico $s \in \Sigma$. Lo stato di un nodo può cambiare nel tempo a seguito di operazioni computazionali o comunicazioni con altri nodi. Non esistono stati intermedi o transitori: un nodo è sempre in uno stato ben definito, e il passaggio da uno stato all'altro avviene in modo discreto, senza stati intermedi.

Definizione 1.1.3 (Evento). In un ambiente distribuito, il comportamento di ogni entità $x \in \mathcal{E}$ è **reattivo**, ovvero è indotto a seguito del verificarsi di **eventi**. In assenza di stimoli, un'entità rimane inerte.

In generale, gli eventi possono essere di vario tipo, come ad esempio:

- **Ricezione di un messaggio**: quando un nodo riceve un messaggio da un altro nodo, questo evento può indurre il nodo a cambiare stato o a eseguire determinate operazioni.
- **Tick del clock**: quando il clock locale di un nodo raggiunge un certo valore, questo evento può indurre il nodo a eseguire determinate operazioni o a cambiare stato.
- **Evento spontaneo**: un nodo può essere indotto a eseguire determinate operazioni o a cambiare stato in modo spontaneo.

Definizione 1.1.4 (Azione). Un' **azione** è una trasformazione che può essere eseguita da un nodo $x \in \mathcal{E}$ in risposta a un evento. Ogni azione modifica lo stato di un nodo o invia un messaggio ad un altro nodo. Formalmente, è una funzione **deterministica** che, fissato un istante di tempo t , mappa tutte le coppie (stato, evento) in un'azione specifica.

$$f : \Sigma \times \hat{E} \rightarrow \mathcal{A}$$

dove $\mathcal{A}$ è l'insieme di tutte le **sequenze di task** che possono essere eseguite, e $\hat{E}$ è l'insieme di tutti gli eventi che possono verificarsi in un ambiente distribuito.

In generale, un'azione può consistere in:

- **computare** un algoritmo;
- **inviare** messaggi ad altri nodi;
- **modificare** lo stato interno del nodo.

In questo modello, ogni azione è **atomica** e **finita**:

- **Atomica**: una volta avviata, un'azione non può essere interrotta;
- **Finita**: un'azione termina sempre in tempo finito.

Si osserva che un'entità in un certo stato potrebbe non reagire a un evento: in questo caso si assume che l'entità abbia eseguito l'azione *nil* (azione nulla).

Definizione 1.1.5 (Comportamento di un'entità). Le associazioni (stato, evento) $\rightarrow$ azione, per ogni nodo $x \in \mathcal{E}$, sono dette **regole**. Definiamo il **comportamento** di un'entità x come l'insieme di tutte le regole che governano le azioni che il nodo esegue in risposta

a eventi specifici, a seconda dello stato in cui si trova. Diremo che il comportamento di un'entità è

- **deterministico** se, per ogni coppia (stato, evento), esiste una regola che associa un'unica azione,
- **completo** se per ogni coppia (stato, evento) esiste almeno una regola che associa un'azione.

Il **comportamento** di un nodo $x \in \mathcal{E}$ è descrivibile mediante l'insieme $B(x)$ che deve essere **non-ambiguo** e **completo**, ossia:

$$\forall (s, e) \in \Sigma \times \hat{E}, \exists! a \in \mathcal{A} : f(s, e) = a$$

Definizione 1.1.6 (Comportamento del Protocollo). Definiamo il **comportamento del protocollo** come l'insieme di tutti i comportamenti dei nodi che partecipano al sistema distribuito. Formalmente è descritto come:

$$B(\mathcal{E}) = \{B(x) : x \in \mathcal{E}\}$$

Un sistema si dice **simmetrico (o omogeneo)** se tutti i nodi condividono lo stesso comportamento, ovvero:

$$B(x) \equiv B(y) \quad \forall x, y \in \mathcal{E}$$

Si nota che un sistema simmetrico è caratterizzato da un protocollo unico che viene eseguito da tutti i nodi. Questo significa che durante l'esecuzione i nodi possono effettuare azioni diverse ma il protocollo deve essere lo stesso. In un sistema asimmetrico (o eterogeneo), l'azione del singolo nodo dipende dalla sua etichetta.

In un sistema **asimmetrico (o eterogeneo)**, il comportamento del singolo nodo dipende dalla sua identificazione o dal suo ruolo. Tuttavia, qualsiasi sistema asimmetrico può essere trasformato in uno simmetrico riscrivendo l'insieme di regole in modo che l'azione eseguita dipenda dalla tipologia del nodo.

Esempio: Consideriamo un sistema con due tipologie di entità: server e client, ognuna con un insieme di regole differenti. È possibile unificare i comportamenti creando un nuovo insieme di regole simmetrico in cui ogni nodo esegue la stessa logica, ma l'azione concreta varia in base al ruolo dell'entità. In questo modo, il protocollo rimane unico e simmetrico per tutti i nodi.

Definizione 1.1.7 (Comunicazione). Le entità in un sistema distribuito interagiscono attraverso lo **scambio di messaggi insieme dei nodi con cui x può comunicare direttamente, ossia:gi (message passing)**. Un messaggio è una **sequenza finita di bit** che transita su canali di comunicazione tra nodi. La topologia di comunicazione è rappresentata da un grafo diretto $G = (V, E)$ ($V = \mathcal{E}$), dove un arco $(x, y) \in E$ indica che il nodo x può inviare messaggi al nodo y . Non tutti i nodi possono comunicare direttamente: la capacità di comunicazione tra due nodi è determinata dalla struttura della rete sottostante.

Si osserva che i nodi *non hanno una visione globale* del sistema (ossia non conoscono la topologia del grafo della rete), ma solo una **visione locale**. Formalmente, $\forall x \in V$, possiamo definire gli insiemi:

- $N_{\text{out}}(x) = \{y \in V : (x, y) \in E\} \subset V$, l'insieme dei nodi a cui x può inviare messaggi (*archi uscenti*);
- $N_{\text{in}}(x) = \{y \in V : (y, x) \in E\} \subset V$, l'insieme dei nodi da cui x può ricevere messaggi (*archi entranti*).

Dunque, definiamo il **vicinato** di un nodo x come

$$N(x) = N_{\text{out}}(x) \cup N_{\text{in}}(x)$$

Definizione 1.1.8 (Tempo di Terminazione). Un protocollo si dice **terminante** se, a partire da qualsiasi configurazione iniziale, raggiunge una configurazione finale in un tempo finito. Formalmente, esiste un tempo finito t tale che, per ogni nodo $x \in V$, il nodo raggiunge uno stato *quiescente* (ossia, uno stato in cui non esegue più azioni) entro t .

Definizione 1.1.9 (Tempo di Completamento). Un protocollo si dice **completante** se, a partire da qualsiasi configurazione iniziale, raggiunge una configurazione finale **accettabile** in un tempo finito. In altre parole, esiste un istante di tempo finito t , per cui il task è risolto, ossia, ogni nodo $x \in V$ si trova in uno stato accettabile entro t .

In generale, un protocollo terminante non è necessariamente completante, e viceversa.

1.2 Assiomi e Restrizioni

La definizione di sistema distribuito è accompagnata da 2 *assiomi fondamentali*. Qualsiasi assunzione aggiuntiva che si vuole fare su un sistema distribuito sarà chiamata *restrizione*

1.2.1 Assiomi

Finite Transmission Delays In assenza di perdite di messaggi, si assume che il tempo di trasmissione di un messaggio tra due nodi sia finito. Ciò implica che, se un nodo invia un messaggio a un altro nodo, il messaggio arriverà entro un tempo finito, anche se questo tempo può essere arbitrariamente lungo, di conseguenza, il sistema non è a conoscenza di questo tempo di trasmissione.

Formalmente, $\forall$ messaggio m inviato da un nodo x a un nodo y , esiste un tempo finito $\tau_{xy} \in \mathbb{R}^+$ tale che m arriva a y entro τ_{xy} .

Local Orientation Questo assioma stabilisce che ogni nodo distingue localmente i suoi vicini, ossia, distingue tra gli archi entranti e quelli uscenti. Un'entità è sempre in grado di inviare un messaggio solamente ad uno specifico e singolo vicino uscente. Di conseguenza, è in grado di distinguere quale dei suoi vicini gli ha inviato un messaggio. Formalmente, ogni nodo x ha una funzione locale iniettiva di orientamento, che assegna un'etichetta univoca a ciascun arco:

$$\lambda_x : E_x \rightarrow L$$

dove $E_x = \{(x, y) \in E\}$ è l'insieme degli archi incidenti su x e L è un insieme di etichette.

Possiamo quindi discutere di grafi etichettati (G, λ) ($\lambda = \{\lambda_x : x \in V\}$), dove ogni arco (**non nodo**) (x, y) ha un'etichetta **locale** $\lambda_x(x, y)$ che identifica univocamente

il vicino y per il nodo x . Allo stesso modo, l'arco (y, x) avrà un'etichetta $\lambda_y(y, x)$ che identifica x per il nodo y .



Figura 1.1: Ciascun arco ha due etichette.

In generale, $\lambda_x(x, y) \neq \lambda_y(x, y)$.

1.2.2 Resitrazioni

Spesso i protocolli vengono definiti e studiati assumendo ulteriori restrizioni. Naturalmente un protocollo definito sotto un gran numero di restrizioni è generalmente debole, nel senso che, rimuovendo alcune di queste restrizioni, il protocollo potrebbe non risolvere il problema preso in analisi. Un protocollo definito su un sistema con poche restrizioni invece risulta più generale, e dunque più forte.

Si elencano alcune tipologie di restrizioni.

Message Ordering Si assume che i messaggi inviati da un nodo x a un nodo y arrivino in ordine FIFO (First In First Out) (**FIFO queue**). Ciò significa che se x invia due messaggi m_1 e m_2 a y , con m_1 inviato prima di m_2 , allora y riceverà m_1 prima di m_2 . Questa restrizione semplifica la progettazione dei protocolli, poiché garantisce un ordine prevedibile dei messaggi tra coppie di nodi.

Bidirectional Links Si assume che le connessioni tra nodi sia *bidirezionale*, ossia, $\forall x, y \in \mathcal{E}$, se $(x, y) \in E$, allora anche $(y, x) \in E$ e inoltre $\lambda_x(x, y) = \lambda_y(y, x)$. Questa restrizione implica che se un nodo x può inviare messaggi a un nodo y , allora y può anche inviare messaggi a x , e che entrambi i nodi condividono la stessa etichetta per identificare la connessione tra di loro. Questa restrizione semplifica la progettazione dei protocolli, poiché garantisce una comunicazione simmetrica tra i nodi. Formalmente,

$$\forall x \in V, N_{\text{out}}(x) = N_{\text{in}}(x) = N(x) \quad \text{e} \quad \lambda_x(x, y) = \lambda_y(y, x) \quad \forall y \in N_{\text{out}}(x)$$

Reliability Restrictions Ci chiediamo: cosa succede al messaggio durante il tragitto? Possiamo fare delle assunzioni sulla disponibilità dei canali di comunicazione.

- **Guaranteed Delivery:** Ogni messaggio che viene inviato viene **ricevuto correttamente**, cioè non ci sono corruzioni di messaggi. Sotto questa restrizione, i protocolli non devono tenere conto di omissioni o corruzioni dei messaggi durante la trasmissione.
- **Partial Reliability:** Non si verificano guasti (ad esempio crash dei nodi, perdita di messaggi) dall'istante in cui si inizia ad osservare il sistema. Si osserva che, negli istanti precedenti all'inizio dell'osservazione, il sistema potrebbe aver subito guasti.

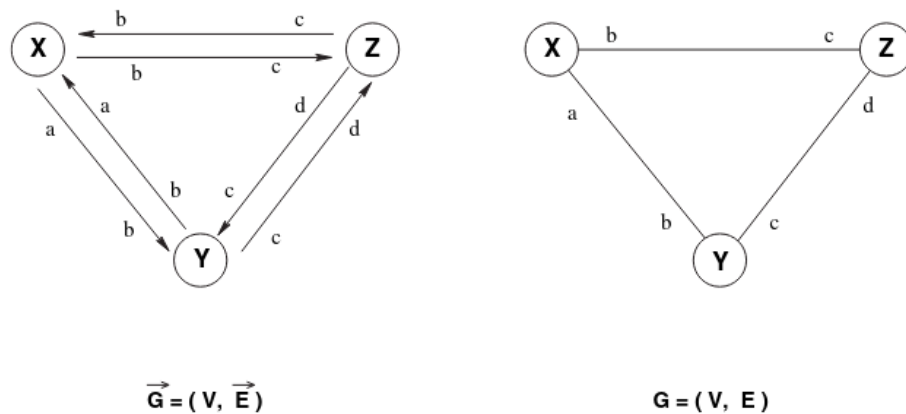


Figura 1.2: In una rete con archi bidirezionali, consideriamo il corrispondente grafo semplificato non diretto.

- **Total Reliability:** Non sono mai avvenuti fallimenti e mai avverranno in futuro.

Topological Restrictions In genere consistono in assunzioni sulla topologia del grafo della rete. Ad esempio assumere che il grafo sia **connesso**/**fortemente connesso**.

Knowledge Restrictions Assumiamo quali sono le informazioni che ogni nodo conosce riguardo la rete. Ad esempio in numero di nodi ($|V| = n$), oppure il numero di archi ($|E| = m$), il diametro della rete, l'etichettamento universale, etc... .

Time Restrictions Eccetto il tempo di trasmissione finito, il modello generale non fa assunzioni su tale quantità. Due possibili restrizioni riguardo i tempi di comunicazione sono le seguenti:

- **Bounded Communication Delay:** Esiste una costante Δ tale che, in assenza di guasti, il delay dei messaggi è al più Δ , in altre parole se un nodo x invia un messaggio m ad un nodo y , m arriverà ad y in al più Δ istanti di tempo.
- **Synchronized Clock:** Tutti i nodi condividono un clock globale sincronizzato, ossia, $\forall x, y \in \mathcal{E}, clock_x(t) = clock_y(t)$ per ogni istante di tempo t . Questa restrizione semplifica la progettazione dei protocolli, poiché consente ai nodi di coordinare le loro azioni basandosi su un riferimento temporale comune.

Cost & Complexity Measures In generale in un sistema distribuito non ha senso parlare di complessità temporale, poiché non è detto che i nodi condividano un clock globale. Per parlare di complessità temporale è necessario assumere la *Synchronized Clock* e la *Bounded Communication Delay*.

Parleremo spesso di **message complexity**, ovvero una misura che tiene conto del numero di messaggi scambiati durante l'esecuzione di un protocollo. Parleremo quindi di numero di bit scambiati, o di numero di messaggi scambiati.

Capitolo 2

Broadcast

Il primo esempio di comunicazione tra processi è il *broadcast*. Consideriamo un sistema distribuito dove un'entità ha una qualche informazione da condividere con tutte le altre entità. Il processo di inviare un messaggio a tutti gli altri processi è chiamato *broadcast*.

Sia $\mathcal{E}$ l'insieme di tutte le entità nel sistema distribuito e sia $G = (V, E)$ il grafo che rappresenta la rete di comunicazione, dove $V = \mathcal{E}$ e E è l'insieme degli archi che rappresentano le interconnessioni tra i nodi.

Prima di procedere con l'algoritmo di broadcast, è importante assumere alcune restrizioni sul sistema distribuito:

Bidirectional Links (BL): Consideriamo un grafo non diretto, dove ogni arco rappresenta una connessione bidirezionale tra due nodi.

Total Reliability (TR): Assumiamo che i messaggi inviati tra i nodi siano sempre consegnati senza errori o perdite.

Connectivity (CN): Il grafo G è connesso, il che significa che esiste un percorso tra ogni coppia di nodi nel grafo. Questa assunzione è fondamentale per garantire la risoluzione del problema di broadcast, poiché se il grafo non fosse connesso, alcuni nodi non potrebbero ricevere il messaggio.

Unique Initiator (UI): Supponiamo che ci sia un unico nodo che inizia il processo di broadcast, chiamato *initiator*. Questo nodo è responsabile di inviare il messaggio iniziale a tutti gli altri nodi.

Formalmente, il problema di broadcast può essere definito come segue: dato un grafo $G = (V, E)$ e un nodo $s \in V$ che è l'*initiator*, l'obiettivo è garantire che ogni nodo $v \in V$ riceva un messaggio da s .

Definiamo l'insieme $\Sigma = \{\text{initiator}, \text{idle}, \text{informed}\}$ come l'insieme di tutti gli stati che può assumere un nodo $x \in \mathcal{E}$.

Definiamo ora le configurazioni del sistema. Una configurazione C è una funzione che associa a ogni nodo $x \in \mathcal{E}$ uno stato in Σ . Inizialmente, la configurazione C_{init} è tale che esiste un unico nodo $s \in V$ tale che $C_{init}(s) = \text{initiator}$ e $C_{init}(x) = \text{idle}$ per ogni nodo

$x \in V \setminus \{s\}$. Il numero di configurazioni iniziali possibili è

$$\binom{n}{1} = n,$$

dove n è il numero di nodi nel grafo. Questo perché possiamo scegliere qualsiasi nodo come initiator.

L'unica configurazione finale accettabile è quella in cui ogni nodo $x \in V$ è nello stato **informed**, ovvero

$$C_{final}(x) = \text{informed} \quad \forall x \in V.$$

Quindi, il problema del broadcast si formalizza mediante la quadrupla:

$$\langle \mathcal{R}, \Sigma, C_{init}, C_{final} \rangle,$$

dove $\mathcal{R} = \{\text{BL}, \text{TR}, \text{CN}, \text{UI}\}$ è l'insieme delle restrizioni, Σ è l'insieme degli stati, C_{init} è la configurazione iniziale e C_{final} è la configurazione finale desiderata.

2.1 Algoritmo Flooding

L'idea generale dell'algoritmo di broadcast è quella di utilizzare un approccio di flooding, dove ogni nodo che riceve il messaggio lo inoltra a tutti i suoi vicini. L'algoritmo può essere descritto come segue:

Algorithm 1 Algoritmo di Broadcast (Flooding)

```

if  $C(s) = \text{initiator}$  then
   $C(s) \leftarrow \text{informed}$ 
  for all  $v \in N(s)$  do
     $\text{send}(m, v)$ 
  end for
end if

if  $C(x) = \text{idle}$  then
   $m \leftarrow \text{receive}()$ 
   $C(x) \leftarrow \text{informed}$ 
  for all  $v \in N(x) \setminus \{\text{sender}\}$  do
     $\text{send}(m, v)$ 
  end for
end if

if  $C(x) = \text{informed}$  then
   $\text{do\_nothing}()$ 
end if

```

Descritto in questa maniera, l'algoritmo di flooding risolve il task nella maniera corretta, e dimostreremo inoltre che termina in un tempo finito. Tuttavia, è importante notare che questo algoritmo non è efficiente in termini di complessità di messaggi, poiché ogni nodo

potrebbe ricevere lo stesso messaggio più volte da diversi vicini, portando a un numero esponenziale di messaggi inviati.

Osservazione. Se non considerassimo l'insieme degli stati $\Sigma = \{\text{initiator}, \text{idle}\}$ e non avessimo la condizione che un nodo non inoltra il messaggio se è già informato, l'algoritmo di flooding potrebbe non terminare, poiché i nodi continuerebbero a inoltrare il messaggio indefinitamente.

Lemma 2.1.1. *L'algoritmo di flooding termina in un tempo finito. Sia $\text{diam}(G)$ il diametro del grafo G , ovvero la massima distanza tra due nodi qualsiasi nel grafo.*

Esiste un tempo massimo τ tale che tutti i nodi x a distanza h da s ricevano il messaggio m entro il tempo τ .

$\forall h \leq \text{diam}(G), \exists \tau \in \mathbb{R}^+ : \forall x \in V \mid \text{dist}(x, s) = h \implies C_{\text{final}}(x) = \text{informed} \wedge t(x) \leq \tau.$

Dimostrazione. Dimostriamo il lemma per induzione su h .

Passo Base $h = 1$ Sfruttando la condizione di Total Reliability, ogni nodo v adiacente a s riceve il messaggio m entro un tempo finito τ_1 . Quindi, per ogni nodo v tale che $\text{dist}(v, s) = 1$, abbiamo $C_f(v) = \text{informed}$ e $t(v) \leq \tau_1$.

Passo Induttivo $h + 1$ Sfruttando la condizione di Connectivity, per ogni nodo $x \in V \setminus \{s\} : d(s, x) = h + 1$.

Questo implica che esiste un shortest path P da $s \rightarrow x$ di lunghezza $h + 1$. Sia y il nodo adiacente a x su questo percorso, quindi $\text{dist}(y, s) = h$. Per ipotesi induttiva, esiste un tempo finito τ_h tale che $C_{\text{final}}(y) = \text{informed}$ e $t(y) \leq \tau_h$.

Sfruttando nuovamente la condizione di Total Reliability, il nodo x riceve il messaggio da y entro un tempo finito τ_{h+1} , quindi $C_{\text{final}}(x) = \text{informed}$ e $t(x) \leq \tau_{h+1}$.

□

Lemma 2.1.2 (Message Complexity). *L'algoritmo di flooding ha una message complexity*

$$M[\text{Flooding}] = \theta(|E|) = \theta(m).$$

Dimostrazione. Dimostriamo il lemma utilizzando due metodi differenti. Nel primo metodo facciamo un'analisi diretta del numero di messaggi inviati, mentre nel secondo metodo utilizziamo un'analisi basata sui nodi e sui loro vicini.

Metodo I: L'idea è di contare il numero di messaggi che attraversano un arco. Se $\text{state}(s) = \text{initiator}$, il numero di messaggi che passano attraverso i suoi archi incidenti è esattamente 1, ossia solo il messaggio m da inoltrare. Consideriamo adesso due nodi $x \neq s$ e $y \neq s$ adiacenti e contiamo il numero di messaggi che passano attraverso l'arco (x, y) . Siano t_x e t_y gli istanti di tempo in cui x e y ricevono il messaggio m , e consideriamo il caso in cui $t_x < t_y$. In questo caso, x inoltra il messaggio m a y . Tuttavia, al tempo t_y , y riceve una copia del messaggio da un nodo $z \neq x$ un'istante prima di riceverlo da x . In questo caso, y inoltra il messaggio a x . Quindi, il numero massimo di messaggi che passano attraverso l'arco (x, y) è 2. Quindi ogni arco può essere attraversato da al massimo 2 messaggi, dimostrando che $M[\text{Flooding}] = \theta(2|E|) = \theta(m).$

Metodo II: Una dimostrazione più precisa può essere ottenuta considerando il numero di messaggi inviati da ogni nodo. Se $state(s) = \text{initiator}$, il numero di messaggi inviati è esattamente $d(s)$ dove $d(x) = N(x)$ è il grado di x . Consideriamo adesso un nodo $x \neq s$. Il numero di messaggi che invia è $d(x) - 1$, poiché non inoltra il messaggio al nodo da cui lo ha ricevuto. Quindi, il numero totale di messaggi inviati è

$$\begin{aligned} M[\text{Flooding}] &= d(s) + \sum_{x \neq s} (d(x) - 1) \\ &= \sum_{x \in V} d(x) - \sum_{x \in V \setminus \{s\}} (n - 1) \\ &= 2|E| - (n - 1) = 2m - n + 1 = \theta(m). \end{aligned}$$

□

Osservazione. La somma degli gradi di tutti i nodi in un grafo è pari a $2|E|$, poiché ogni arco contribuisce al grado di due nodi.

2.1.1 Complessità Temporale

In generale, non ha senso calcolare la complessità temporale di un algoritmo in un sistema asincrono a causa dell'impossibilità di poter predire il tempo di trasmissione dei messaggi. Tuttavia, se assumiamo un sistema sincrono, ossia assumiamo **clock sincronizzati** e **bounded delay**, possiamo calcolare la complessità temporale dell'algoritmo di flooding. Assumiamo che i messaggi vengano trasmessi con un delay di δ (normalizzando, $\delta = 1$). Chiaramente, il messaggio inviato dall'initiator s deve raggiungere ogni nodo x a distanza h da s entro un tempo finito, e il tempo necessario per raggiungere il nodo più lontano da s è dato **eccentricity** di s , che è definita come

$$ecc(s) = \max_{x \in V} dist(s, x).$$

Allora, nel caso peggiore, il tempo necessario per raggiungere ogni nodo è dato dal diametro del grafo $\text{diam}(G)$, che è definito come

$$\text{diam}(G) = ecc(s).$$

In conclusione, la complessità temporale *ideale* dell'algoritmo di flooding è

$$T[\text{Flooding}] = \max_{x \in V} dist(s, x) \leq \text{diam}(G) \leq n - 1 = O(n).$$

Si vuole dimostrare inoltre che tale protocollo sia ottimale in termini di complessità temporale, ossia che non esiste un protocollo di broadcast che possa raggiungere ogni nodo in un tempo minore del diametro del grafo. È necessario quindi dare una delimitazione inferiore alla complessità temporale di qualsiasi protocollo di broadcast. Si osserva che ciascuna entità x a distanza h da s non può ricevere il messaggio m prima di h unità di tempo, poiché il messaggio deve attraversare almeno h archi per raggiungere x . Quindi, la complessità temporale di qualsiasi protocollo di broadcast è limitata inferiormente da

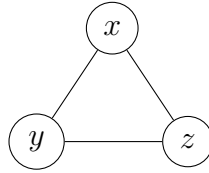
$$T[\text{Broadcast}] \geq \max_{x \in V} dist(s, x) = ecc(s) \leq \text{diam}(G).$$

Di conseguenza, l'algoritmo di flooding è ottimale in termini di complessità temporale, in particolare l'ideal time complexity è $T[\text{Flooding}] = \theta(\text{diam}(G))$.

2.1.2 Stati ed Eventi

Nel mondo centralizzato, dato uno stesso input ad un algoritmo deterministico, esso non solo genera lo stesso output, ma la sequenza di azioni che svolge per raggiungere tale output è sempre la stessa, ad ogni esecuzione. Questo dettaglio non vale nel mondo distribuito. Ad esempio nel protocollo flooding, dato che il ritardo dei messaggi varia in modo indeterminato, la sequenza di azioni verso l'unica configurazione finale varia ad ogni esecuzione, anche a fronte di una stessa configurazione iniziale (cioè esecuzioni dallo stesso iniziatore). Naturalmente, per la correttezza del protocollo, una qualsiasi sequenza di azioni anche se diversa converge verso la configurazione finale.

Consideriamo il seguente grafo G con $n = 3$ nodi e $m = 3$ archi, dove x è l'iniziatore:



Eseguendo una prima volta il protocollo, supponiamo che z riceva il messaggio da x prima di y .

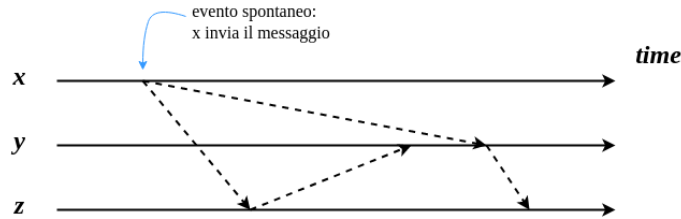
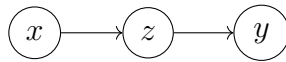


Figura 2.1: Sequenza di azioni del protocollo di flooding, con z che riceve il messaggio da x prima di y .

Tale esecuzione può essere rappresentata mediante il seguente grafo, dove ogni nodo rappresenta una configurazione del sistema e ogni arco rappresenta un evento (cioè, l'invio o la ricezione di un messaggio). In questo caso, la sequenza di azioni è $x \rightarrow z \rightarrow y$.

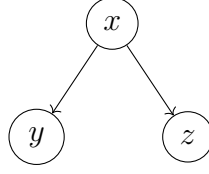


Eseguendo una seconda volta il protocollo, potrebbe capitare che i ritardi sugli archi (x, z) e (z, y) siano simili. Perciò, una possibile sequenza di azione potrebbe essere:



Figura 2.2: Esecuzione del protocollo con ritardi simili sugli archi (x, z) e (z, y) .

Tale esecuzione può essere rappresentata mediante il seguente grafo,



Dato che il grafo di comunicazione è sostanzialmente un albero ricoprente, ci sono quindi $O(n^n)$ possibili sequenze di trasmissioni (o **traiettorie**) che portano alla configurazione finale, anche a fronte di una stessa configurazione iniziale.

2.1.3 Lowerbound alla Message Complexity

Definizione 2.1.1 (Stato Locale Interno). Definiamo $\sigma(x, t)$ come lo stato locale interno del nodo x al tempo t . Di conseguenza

$$\Sigma(t) = \{\sigma(x, t) : x \in \mathcal{E}\}$$

è l'insieme di tutti gli stati locali dei nodi al tempo t .

Consideriamo adesso due ambienti (*environments*) E_1 e E_2 dove, lo stato interno di x al tempo t è lo stesso, ossia $\sigma_1(x, t) = \sigma_2(x, t)$. Allora x non è in grado di *distinguere* tra i due ambienti. Di conseguenza, se avviene lo stesso evento in entrambi gli ambienti, x esegue la stessa azione in entrambi gli ambienti.

Seguono due proprietà fondamentali che ci permettono di dimostrare un lowerbound alla message complexity di qualsiasi protocollo di broadcast.

Proprietà 1: Se x si trova in due ambienti E_1 e E_2 , e lo stato interno di x è lo stesso in entrambi gli ambienti al tempo t ($\sigma_1(x, t) = \sigma_2(x, t)$), allora x non è in grado di distinguere tra i due ambienti.

Proprietà 2: Se due nodi distinti x e y ad un certo istante t si trovano nello stesso stato interno ($\sigma(x, t) = \sigma(y, t)$), allora se vengono sollecitati dallo stesso evento, eseguiranno la stessa azione.

Teorema 2.1.3 (Lower Bound). *Sotto le ipotesi*

$$R = \{BL, TR, CN, UI\}$$

(ossia, *bidirectional links, total reliability, connectivity e unique initiator*), qualsiasi protocollo di broadcast ha una message complexity $M[\text{Broadcast}] = \Omega(m)$.

Dimostrazione. Supponiamo per assurdo che esista un protocollo di broadcast con una message complexity $M[\text{Broadcast}] = o(m)$ (per esempio $m - 1$ messaggi). Consideriamo un grafo $G = (V, E)$, e sia s l'initiator. Poiché il protocollo ha una message complexity sublineare, esiste almeno un arco $e = (x, y) \in E$ che non viene attraversato da alcun messaggio durante l'esecuzione del protocollo. Sia ora $G' = (V', E')$ costruito a partire da G rimuovendo l'arco e e aggiungendo un nuovo nodo z tale per cui $state(z) = \text{idle}$ connesso a x e y mediante due nuovi archi, ossia

$$V' = V \cup \{z\}, \quad E' = E \setminus \{e\} \cup \{(x, z), (y, z)\}.$$

Siano quindi E_1 e E_2 i due ambienti, dove in E_1 il protocollo viene eseguito su G , mentre in E_2 il protocollo viene eseguito su G' . Le etichette degli archi in G e G' sono le stesse,

quindi x e y non sono in grado di distinguere tra i due ambienti.

$$\lambda_x(x, y) = \lambda_x(x, z), \quad \lambda_y(y, x) = \lambda_y(y, z).$$

Allora sapendo che x e y non sono in grado di distinguere (**Proprietà 1**) tra i due

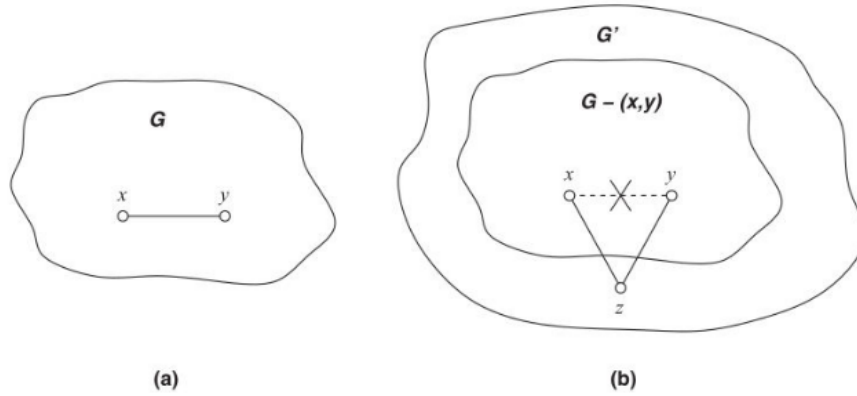


Figura 2.3: Costruzione del grafo G' a partire da G .

ambienti, se viene sollecitato lo stesso evento in entrambi gli ambienti, x e y eseguono la stessa azione (**Proprietà 2**). Poiché in E_1 non inoltrano il messaggio sul arco (x, y) , in E_2 non inoltrano il messaggio né su (x, z) né su (y, z) . Di conseguenza, il nodo z non riceve mai il messaggio (poiché è collegato solo a x e y), e quindi il protocollo di broadcast fallisce in E_2 , portando ad una contraddizione, ossia vengono inviati $o(m)$ messaggi e rimane un nodo non raggiungibile. $\square$

Osservazione. Se consideriamo una fase di preprocessing, ossia una fase iniziale in cui i nodi possono scambiarsi messaggi per costruire una mappa della rete (uno **spanning tree**), allora è possibile progettare un protocollo di broadcast con una message complexity $M[\text{Broadcast}] = O(n)$, che è ottimale in quanto ogni nodo deve ricevere almeno un messaggio. Tuttavia, in questo caso, la message complexity totale del protocollo (preprocessing + broadcast) è $M[\text{Total}] = O(n + m) = O(m)$, che è ancora ottimale in quanto ogni arco deve essere attraversato da almeno un messaggio durante la fase di preprocessing.

Capitolo 3

Spanning Tree Construction

Definizione 3.0.1 (Spanning Tree). Un albero ricoprente (Spanning Tree) di un grafo $G = (V, E)$ è un sottografo connesso e aciclico di G tale che $T = (V, E')$ dove $E' \subseteq E$. In altre parole è un albero che include tutti i nodi di V .

Definizione 3.0.2 (SPT Construction Distributed Problem). In un sistema distribuito, il problema di costruire un albero ricoprente (Spanning Tree) del grafo $G = (V, E)$ significa portare il sistema da uno stato iniziale, in cui ogni nodo è consapevole solo di se stesso e dei suoi vicini, a uno stato finale in cui tutti i nodi sono organizzati in una struttura ad albero che copre tutti i nodi del grafo. In altre parole,

Configurazione Iniziale: Ciascun nodo $x \in \mathcal{E} = V$ mantiene una struttura dati $\text{tree_neigh}(x) = \emptyset$. La possiamo immaginare come un vettore binario, dove ogni posizione rappresenta un nodo vicino ad x , e il valore 1 indica che l'arco tra x e quel nodo è parte dell'albero, mentre 0 indica che non lo è.

Configurazione Finale: $\forall x \in \mathcal{E} = V$,

$$\text{tree_neigh}(x) = \{\text{archi che appartengono allo ST}\},$$

e l'unione di tutti questi insiemi per forma un albero ricoprente del grafo G .

Il problema è formalizzato dalla tripla $P = \langle P_{init}, P_{final}, R \rangle$, dove:

$$P_{init} = \forall x \in V,$$

$$\begin{aligned} \text{tree_neigh}(x) &= \emptyset \\ \exists! s \in V, \text{state}(s) &= \text{initiator} \\ \forall x \in V \setminus \{s\}, \text{state}(x) &= \text{idle} \end{aligned}$$

$$P_{final} = \forall x \in V,$$

$$\begin{aligned} \text{tree_neigh}(x) &= \{y \in N(x) : (x, y) \in E'\} \\ \bigcup_{x \in V} \text{tree_neigh}(x) &= T(V, E') \text{ albero ricoprente del grafo } G \\ \forall x \in V, \text{state}(x) &= \text{done} \end{aligned}$$

$$R = \{\text{TR}, \text{BL}, \text{CN}, \text{UI}\}$$

Quindi, a differenza di un algoritmo centralizzato (Kruskal, Prim), la conoscenza dello ST è distribuita tra i nodi, e ogni nodo deve collaborare con i suoi vicini per costruire l'albero ricoprente. Non esiste quindi una visione globale dell'albero, ma ogni nodo deve basarsi solo sulle informazioni locali e sui messaggi ricevuti dai suoi vicini per decidere se includere o meno un arco nel suo insieme $\text{tree_neigh}(x)$.

3.1 Single Source SHOUT Protocol

L'idea dell'algoritmo è quella di chiedere ai nodi vicini se sono interessati a far parte dello ST, e se sì, includere l'arco che li collega al nodo che ha fatto la richiesta. In questo modo, partendo da un nodo iniziale (initiator), si costruisce gradualmente lo ST coinvolgendo i nodi vicini.

Definiamo $\Sigma = \{\text{initiator}, \text{idle}, \text{active}, \text{done}\}$ dove $S_{\text{init}} = \{\text{initiator}, \text{idle}\}$ l'insieme dei stati iniziali e $S_{\text{final}} = \{\text{done}\}$ l'insieme dei stati finali.

- Inizialmente, tutti i nodi tranne l'iniziatore s sono in stato **idle**, e s è in stato **initiator**. Spontaneamente, s si sveglia e invia i messaggi di domanda (Q) a tutti i suoi vicini, proponendo loro di essere connessi a lui nello ST (cioè essere suoi figli).
- Quando un nodo x nello stato **idle** riceve un messaggio Q, questo si attiva, sceglie come padre il **primo** nodo da cui ha ricevuto il messaggio e invia a tale nodo il messaggio YES.
- Successivamente, y invia a tutti i suoi vicini, tranne il padre, un messaggio Q per chiedere se vogliono essere suoi figli. Se un nodo z nello stato **active** riceve un messaggio Q da y , risponde con un messaggio NO perché ha già scelto un padre e non può accettare più di uno. Se invece z è nello stato **idle**, risponde con un messaggio YES e si attiva, scegliendo y come padre.
- Ogni nodo conta il numero di risposte ricevute da suoi vicini. Quando un nodo x ha ricevuto risposte da tutti i suoi vicini, passa allo stato **done** e non fa più nulla.

Per un nodo x , i vicini che rispondono con YES saranno inclusi nel suo insieme $\text{tree_neigh}(x)$ includendo anche il vicino che ha fatto la richiesta.

Possiamo quindi considerare il protocollo SHOUT come un broadcast con reply.

Algorithm 2 Algoritmo di SHOUT

```
if state(x) = initiator then spontaneously
    root  $\leftarrow$  true
    tree_neigh(x)  $\leftarrow$   $\emptyset$ 
    state(x)  $\leftarrow$  active
    counter  $\leftarrow$  0
    for all  $v \in N(x)$  do
        send(Q, v)
    end for
end if

if state(x) = idle then
    Q  $\leftarrow$  receive()
    tree_neigh(x)  $\leftarrow$  {sender}
    root  $\leftarrow$  false
    counter  $\leftarrow$  1
    send(YES, sender)
    if counter =  $|N(x)|$  then
        state(x)  $\leftarrow$  done
    else
        for all  $v \in N(x) \setminus \{\text{sender}\}$  do
            send(Q, v)
        end for
        state(x)  $\leftarrow$  active
    end if
end if

if state(x) = active then
    reply  $\leftarrow$  receive()
    if reply = YES then
        tree_neigh(x)  $\leftarrow$  tree_neigh(x)  $\cup$  {sender}
        counter  $\leftarrow$  counter + 1
        if counter =  $|N(x)|$  then
            state(x)  $\leftarrow$  done
        end if
    end if
end if

    if reply = NO then
        counter  $\leftarrow$  counter + 1
        if counter =  $|N(x)|$  then
            state(x)  $\leftarrow$  done
        end if
    end if

    if reply = Q then
        send(NO, sender)
    end if
end if
```

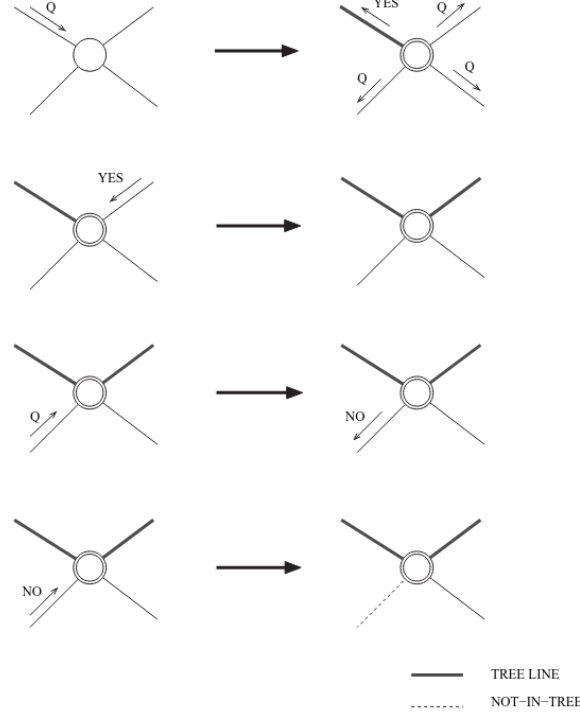


Figura 3.1: Insieme di regole del protocollo SHOUT.

3.1.1 Terminazione

Il protocollo SHOUT è molto simile al protocollo di broadcast con l'aggiunta dei messaggi di reply YES e NO. Poiché siamo sotto le assunzioni TR e CN, allora ogni nodo $x \neq s$ riceverà entro un tempo finito un messaggio Q da un vicino, e quindi si attiverà impostando il contatore a 1 (per la correttezza del protocollo di broadcast). Per ogni richiesta ricevuta, i nodi risponderanno con un messaggio YES o NO, di conseguenza, ogni nodo riceverà (per la TR e per la costruzione del protocollo, ogni nodo risponderà a tutte le richieste) un numero finito di risposte ($|N(x)|$), e quindi, entro un tempo finito, ogni nodo raggiungerà lo stato done, terminando localmente la propria esecuzione.

3.1.2 Correttezza

Per dimostrare la correttezza del protocollo SHOUT, dobbiamo mostrare che alla fine dell'esecuzione, l'insieme di archi scelti da tutti i nodi forma un albero ricoprente del grafo G .

Lemma 3.1.1 (Coerenza). *Siano x e y due nodi vicini in G . Se x ha come vicino y in ST , cioè $y \in \text{tree_neigh}(x)$, allora x è vicino di y in ST , cioè $x \in \text{tree_neigh}(y)$. Quindi*

$$\forall x, y \in V, y \in \text{tree_neigh}(x) \iff x \in \text{tree_neigh}(y).$$

Dimostrazione. Supponiamo che $y \in \text{tree_neigh}(x)$. Allora y ha risposto con un messaggio YES alla richiesta di x , e quindi y ha scelto x come padre. Ma se y ha risposto con YES a x , allora y era nello stato idle quando ha ricevuto la richiesta da x e per definizione del protocollo y inserisce x nel suo insieme $\text{tree_neigh}(y)$. Quindi $x \in \text{tree_neigh}(y)$. La dimostrazione dell'implicazione inversa è simile, con x e y scambiati. $\square$

Lemma 3.1.2 (Connettività/Spanning). *Il sottografo $T = (V, E')$ definito da $E' = \bigcup_{x \in V} \text{tree_neigh}(x)$ è connesso e include tutti i nodi di V . In altre parole, il sottografo indotto dagli archi su cui è viaggiato il messaggio YES è connesso e include tutti i nodi di V (è ricoprente e connesso).*

Dimostrazione. Vogliamo dimostrare che per ogni nodo $x \in V \setminus \{s\}$, se x ha risposto YES ad un nodo y , allora $x \in \text{tree_neigh}(y)$, dove quest'ultimo è connesso alla sorgente s tramite una sequenza di archi sui quali sono passati i messaggi YES, e tale sequenza formerà un cammino da $s \rightarrow x$.

Sia x un nodo qualsiasi in $V \setminus \{s\}$. Se x ha risposto YES ad un nodo y , allora $x \in \text{tree_neigh}(y)$, e dato che y deve aver inviato una proposta al nodo x , allora y si trova in **active** alla ricezione della risposta YES da x . Ma y invia una proposta al nodo x solo se:

- y è la sorgente s , e quindi è connesso a se stesso, oppure
- y ha ricevuto una proposta da un nodo z e ha risposto YES a z , e quindi y è connesso a z tramite un arco su cui è passato un messaggio YES, quindi

$$z \in \text{tree_neigh}(y) \text{ e } y \in \text{tree_neigh}(z) \quad \text{Lemma 3.1.1}$$

Iterando questo processo fino alla sorgente s , si ottiene un cammino che collega s a x tramite archi su cui sono passati messaggi YES, e quindi x è connesso a s . □

Lemma 3.1.3 (Aciclicità). *Il sottografo $T = (V, E')$ definito da*

$$E' = \bigcup_{x \in V} \text{tree_neigh}(x) \text{ è aciclico.}$$

Dimostrazione. Supponiamo per assurdo che T contenga un ciclo. Allora esiste una sequenza di nodi $v_1, v_2, \dots, v_k$ con $k \geq 3$ tale che $(v_i, v_{i+1}) \in E'$ (ossia v_i è padre di v_{i+1}) per ogni $i = 1, 2, \dots, k-1$ e $(v_k, v_1) \in E'$.

Poichè ogni nodo, eccetto la sorgente invia **un solo** messaggio YES, allora il ciclo $v_1, v_2, \dots, v_k$ non contiene la sorgente s . Per il [Lemma 3.1.2](#), deve esistere un cammino da s a qualsiasi nodo, inclusi quelli del ciclo. Ma poiché deve esistere un cammino da s a v_i , allora esiste un cammino da s a v_i che passa attraverso il ciclo, il che implica che esiste un cammino da s a v_i che contiene un ciclo. Questo è una contraddizione, poiché un cammino non può contenere un ciclo. Quindi, il sottografo T deve essere aciclico. □

Teorema 3.1.4. *Il protocollo SHOUT costruisce un albero ricoprente del grafo G .*

Dimostrazione. Dal [Lemma 3.1.2](#) sappiamo che il sottografo T è connesso e include tutti i nodi di V , e dal [Lemma 3.1.3](#) sappiamo che T è aciclico. Quindi, combinando questi due risultati, possiamo concludere che T è un albero ricoprente del grafo G . □

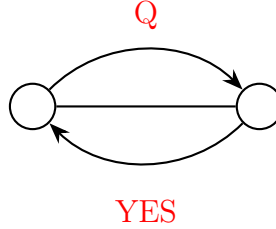
3.1.3 Message Complexity

Come menzionato sopra, il protocollo SHOUT = Flood + Reply. Quindi la complessità in termini di messaggi consiste nell'esecuzione di Flood(Q) più le risposte YES o NO per ogni richiesta ricevuta. In altre parole,

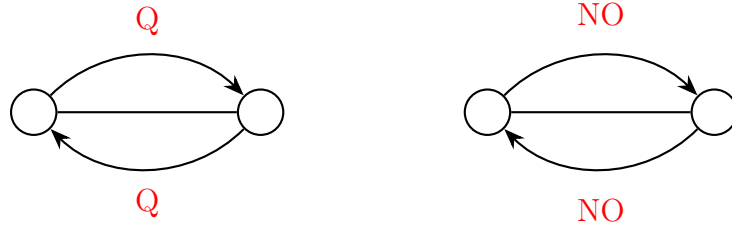
$$M[\text{SHOUT}] = M[\text{Flood}(Q) + \text{Reply}] = 2M[\text{Flood}(Q)] = O(4|E|) = O(|E|).$$

Più precisamente, i messaggi inviati dal protocollo possono essere divisi in due categorie: Q e Reply = { YES, NO }. Di conseguenza, abbiamo le seguenti casistiche:

- Per ogni arco $(x, y) \in ST$ costruito dal protocollo passano due messaggi: un messaggio Q inviato da x a y e un messaggio YES inviato da y a x . Il numero di archi sono $|E'| = |V| - 1$, quindi il numero di messaggi in questa categoria è $2(|V| - 1) = 2(n - 1)$.



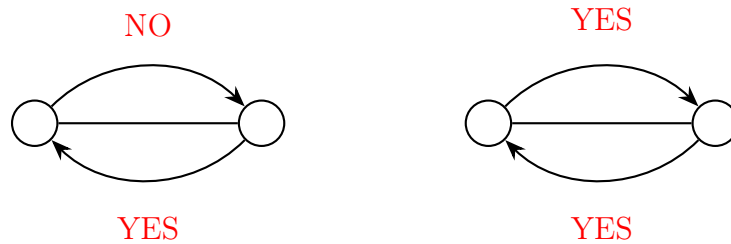
- Per ogni arco $(x, y) \in E \setminus ST$ passano quattro messaggi: due messaggi Q in entrambe le direzioni, seguiti da due messaggi NO in entrambe le direzioni. Il numero di archi in questa categoria è $|E| - |E'| = |E| - (|V| - 1)$, quindi il numero di messaggi in questa categoria è $4(|E| - (|V| - 1)) = 4(m - n + 1)$.



Osservazione. Su ogni arco (x, y) non potranno mai passare le coppie di messaggi (NO, YES) e (YES, YES).

(NO, YES) Non può accadere in quanto il nodo sinistro per rispondere a NO deve prima aver ricevuto una richiesta dal nodo destro, ma per inviare una richiesta, il nodo destro deve avere già un padre, dunque non può rispondere con YES a nessuna richiesta.

(YES, YES) Non può accadere in quanto due nodi per inviarsi reciprocamente un messaggio YES devono essersi entrambi inviati reciprocamente un messaggio Q, ma ciò implica che hanno già un padre, e quindi hanno già risposto con un messaggio YES a un altro nodo.



Quindi, il numero totale di messaggi è:

$$M[\text{SHOUT}] = 2(n - 1) + 4(m - n + 1) = 4m - 2n + 2 = O(|E|).$$

3.2 Single Source SHOUT+ Protocol